

# Frontend Architecture: NetOps AI NOC Dashboard

## What frontend does

A real-time web dashboard for network teams to:

- monitor devices live
  - see alerts instantly
  - inspect device health
  - detect config drift
  - run automation workflows
  - view incidents
  - generate reports
  - track reliability/SLA
- 

## High-Level Frontend Architecture

React / Next.js Frontend



API Client Layer



FastAPI Backend APIs



WebSocket Client



Live Updates: devices, alerts, incidents



Dashboard UI Pages

---

## Recommended Frontend Stack

Area

Tech

|                |                         |
|----------------|-------------------------|
| Framework      | React + Vite or Next.js |
| Language       | TypeScript              |
| Styling        | Tailwind CSS            |
| UI Components  | shadcn/ui               |
| Charts         | Recharts                |
| Tables         | TanStack Table          |
| State          | Zustand                 |
| API Calls      | Axios / React Query     |
| Real-time      | WebSocket               |
| Topology Graph | React Flow              |
| Maps optional  | Mapbox / Leaflet        |

---

# Main Frontend Pages

## 1. Dashboard Overview

Shows:

- total devices
- online/offline count
- active alerts
- open incidents
- average latency
- reliability score
- live telemetry charts

Cards + charts + live incident feed

---

## 2. Network Topology Page

Visual graph of:

- routers
- switches
- access points
- edge nodes
- failed links
- degraded devices

Use:

React Flow

This makes the project look enterprise-level.

---

### 3. Device Inventory Page

Table of all devices:

- device name
- type
- IP address
- location
- health score
- status
- last heartbeat

Actions:

- view details
  - run diagnostics
  - check config
  - trigger recovery
- 

### 4. Device Detail Page

For each device:

- CPU chart
- memory chart
- latency chart
- packet loss chart

- bandwidth chart
  - recent alerts
  - config status
  - automation history
- 

## 5. Alerts Page

Shows real-time alerts:

- severity
- device
- reason
- timestamp
- status
- suggested action

Severity levels:

INFO / WARNING / CRITICAL

---

## 6. Incident Management Page

Shows:

- open incidents
- affected devices
- timeline
- root cause
- automation steps
- resolution status

Incident lifecycle:

Open → Investigating → Mitigated → Resolved

---

## 7. Config Management Page

Shows:

- baseline config
- current config
- drift status
- config diff
- version history
- rollback button

Very important for recruiter match because JD mentions **device configuration**.

---

## 8. Automation Workflows Page

Shows:

- available workflows
- previous runs
- success/failure logs
- recovery actions

Example workflows:

- restart device agent
  - rollback config
  - isolate degraded node
  - create incident
  - send notification
- 

## 9. Reports Page

Generate:

- uptime report
- SLA report
- incident report
- config compliance report
- reliability summary

Export:

PDF / CSV / JSON

---

## 10. Settings/Admin Page

Manage:

- users
  - roles
  - alert thresholds
  - notification settings
  - device groups
- 

## Frontend Data Flow

User opens dashboard  
↓  
Frontend calls REST API for initial data  
↓  
Frontend opens WebSocket connection  
↓  
Backend pushes live telemetry/alerts  
↓  
Zustand updates global state  
↓  
UI updates instantly

---

## API Integration Plan

### REST APIs

Use REST for:

GET /devices  
GET /devices/{id}  
GET /incidents  
GET /alerts  
GET /configs/{device\_id}  
POST /automation/run  
GET /reports

# WebSocket APIs

Use WebSocket for:

/ws/telemetry

/ws/alerts

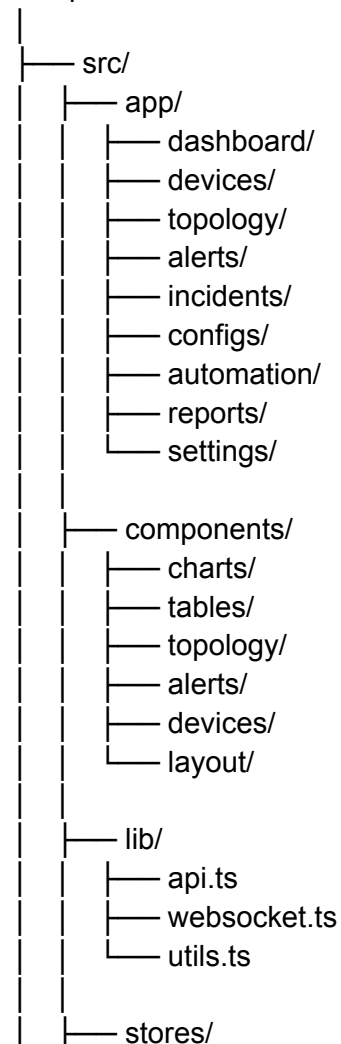
/ws/incidents

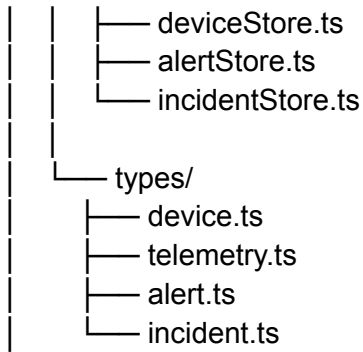
/ws/device-status

---

# Frontend Folder Structure

netops-ai-frontend/





## Best UI Layout

Left Sidebar:

- Overview
- Topology
- Devices
- Alerts
- Incidents
- Configs
- Automation
- Reports

Top Bar:

- Search
- Live status
- Notification bell
- User profile

Main Area:

- Cards
  - Charts
  - Tables
  - Graphs
- 

## MVP Frontend Build Order

### Phase 1



Build:

- layout/sidebar
- dashboard overview
- device inventory
- static charts with mock data

## Phase 2

Connect:

- FastAPI REST APIs
- real device data
- telemetry history

## Phase 3

Add:

- WebSocket live updates
- alerts feed
- status changes

## Phase 4

Add:

- topology graph
- device details
- incidents

## Phase 5

Add:

- config drift UI
  - automation workflow UI
  - reports export
-

# Best Recruiter-Friendly Final Architecture

Device Simulators



Redis Streams / Kafka



FastAPI Backend



PostgreSQL + TimescaleDB + Redis



REST APIs + WebSockets



React TypeScript Dashboard



NOC-style Monitoring UI

## Final Project Description

**NetOps AI** is a real-time network monitoring and automation platform that simulates enterprise routers, switches, access points, and edge nodes, streams telemetry into a Python FastAPI backend, detects anomalies, manages configurations, triggers automated recovery workflows, and visualizes the entire network through a React-based NOC dashboard.